

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

Advanced Programming (OOPs) Lab

Spring 2026

Modular Library Management System with Fine Calculation and Reservation Engine

A Java-Based OOP Project Report

Submitted By
ADRIJA DUTTA

Roll Number: 73

Department: CSE(IoTCSBT)

Enrolment Number: 12024052017063

Course & Semester: B.Tech. 4th Semester

Academic Year: 2025-2026

Subject Code: PCCCS495

Instructors: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas



1. Problem Statement

Traditional library management systems often rely on manual record-keeping or monolithic software that is difficult to extend, maintain, or debug. As library collections and membership grow, managing book availability, tracking borrowing histories, and enforcing lending policies for different user categories becomes increasingly error-prone when done manually.

This project addresses those challenges by designing and implementing a Modular Library Management System in Java. The system applies core Object-Oriented Programming principles to build a clean, extensible architecture that supports two types of library members — students and faculty — each with distinct borrowing rules. It provides functionalities for issuing and returning books, calculating fines based on overdue periods, handling edge cases through a custom exception hierarchy, and persisting data across sessions using Java serialization.

The system targets library administrators who need a reliable, rule-enforcing platform that can easily be extended to accommodate new member types, book reservation queues, or fine structures without modifying existing logic.

2. System Design

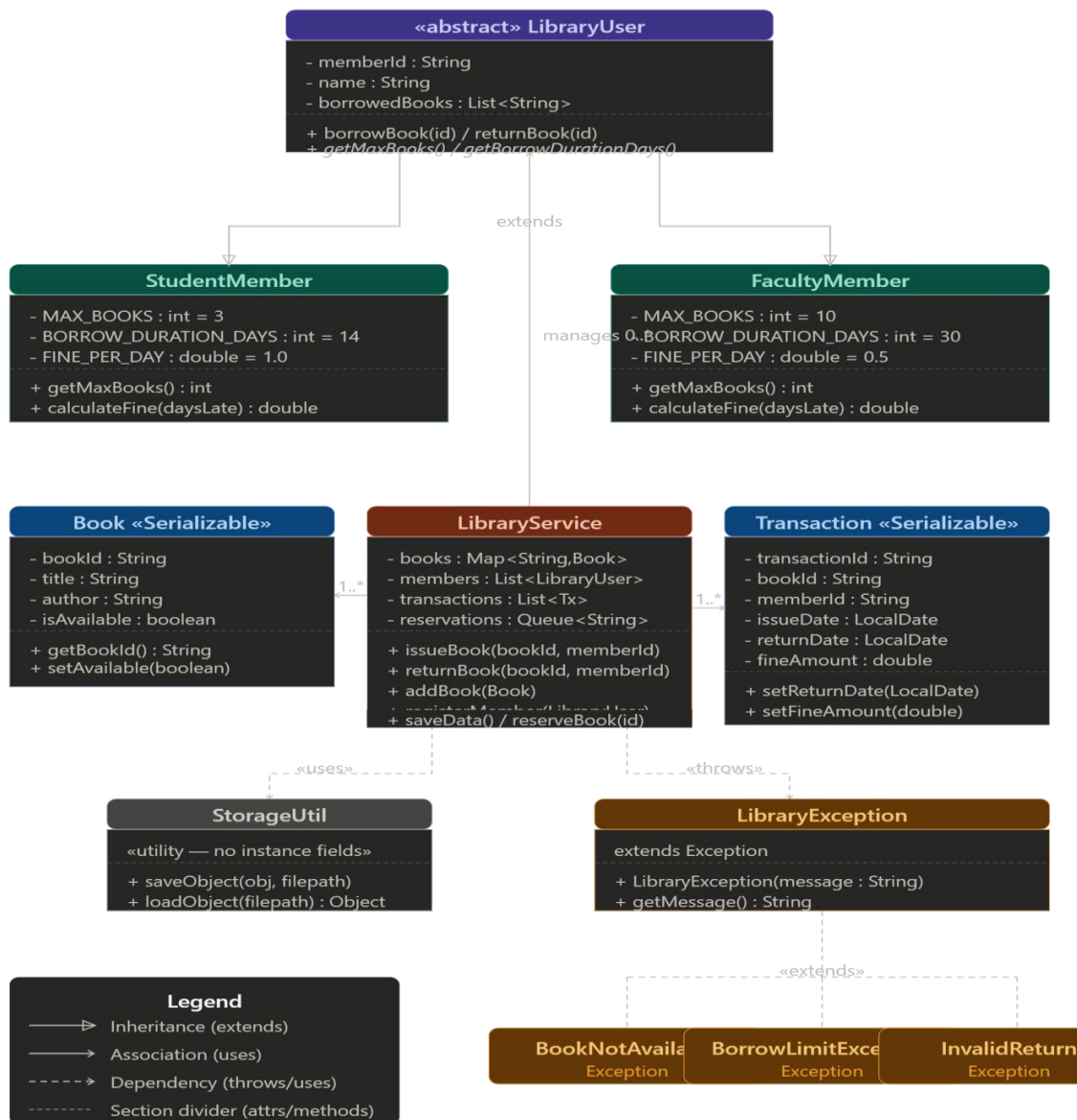
2.1 Package Structure

The codebase is organised into four packages, each with a clear responsibility boundary:

- `model` — Domain entities: `Book`, `LibraryUser` (abstract), `StudentMember`, `FacultyMember`, `Transaction`.
- `service` — Business logic: `LibraryService` orchestrates all operations (issue, return, reserve, persist).
- `exception` — Custom exceptions: `LibraryException` (base), `BookNotAvailableException`, `BorrowLimitExceededException`, `InvalidReturnException`.
- `util` — Cross-cutting concern: `StorageUtil` provides object serialisation and deserialisation helpers.
- `(root)` — Entry point: `Main.java` drives the CLI interaction loop.

2.2 UML Class Diagram

The diagram below illustrates the relationships among the major classes in the system.



2.3 Interaction Flow

A typical issue-book interaction follows this sequence:

- User enters option 3 in the CLI; Main.java reads bookId and memberId.
- Main.java calls LibraryService.issueBook(bookId, memberId).
- LibraryService looks up the Book in the books map and the LibraryUser in the members list.
- If the book is unavailable, BookNotAvailableException is thrown and propagated to Main.
- If the member has reached their borrow limit, BorrowLimitExceededException is thrown.
- Otherwise, book.setAvailable(false), member.borrowBook(bookId), a new Transaction is created, and saveData() is called.
- StorageUtil serialises the updated books, members, and transactions maps to disk.

3. OOP Concepts Demonstrated

3.1 Abstraction

Where: The abstract class `LibraryUser` defines the contract that all library members must fulfil, without exposing implementation details to external callers.

Why: Different member types (students, faculty) share core state (`memberId`, `name`, `borrowedBooks`) but differ in borrowing limits, durations, and fine rates. Abstraction captures the shared contract while delegating specifics to subclasses, keeping `LibraryService` free of conditionals like `if(student) ... else if(faculty)`.

Code Snippet (`LibraryUser.java`):

```
// Abstract class - defines contract, hides implementation
public abstract class LibraryUser implements Serializable {
    private String memberId;
    private List<String> borrowedBooks = new ArrayList<>();

    // Abstract methods: subclasses supply policy
    public abstract int getMaxBooks();
    public abstract int getBorrowDurationDays();
    public abstract double calculateFine(int daysLate);

    // Concrete shared behaviour
    public void borrowBook(String bookId) {
        this.borrowedBooks.add(bookId);
    }
}
```

Explanation: The three abstract methods form a policy interface. `LibraryService` calls `member.getMaxBooks()` and `member.calculateFine(daysLate)` without ever knowing whether it is dealing with a student or a faculty member. The concrete state (`memberId`, `borrowedBooks`) lives in the abstract class because it is shared by all member types.

3.2 Inheritance

Where: `StudentMember` and `FacultyMember` both extend `LibraryUser` and inherit its state and concrete methods while overriding the three abstract policy methods.

Why: Inheritance eliminates code duplication. Both subclasses need `memberId`, `name`, `borrowedBooks`, `borrowBook()`, and `returnBook()`, which are provided by the parent. Only the policy constants differ, so only those are overridden.

Code Snippet (`StudentMember.java`):

```
public class StudentMember extends LibraryUser {
    private static final int MAX_BOOKS = 3;
    private static final int BORROW_DURATION_DAYS = 14;
    private static final double FINE_PER_DAY = 1.0;

    public StudentMember(String memberId, String name) {
        super(memberId, name); // delegates to parent constructor
    }

    @Override public int getMaxBooks() { return MAX_BOOKS; }
    @Override public int getBorrowDurationDays() { return BORROW_DURATION_DAYS; }
    @Override public double calculateFine(int daysLate) {
        return daysLate <= 0 ? 0.0 : daysLate * FINE_PER_DAY;
    }
}
```

Explanation: The call to `super(memberId, name)` delegates initialisation of the shared fields to the parent class. The three `@Override` methods supply the student-specific policy values. `FacultyMember`

follows the identical pattern with `MAX_BOOKS=10`, `BORROW_DURATION_DAYS=30`, and `FINE_PER_DAY=0.5`, demonstrating how inheritance promotes reuse while allowing specialisation.

3.3 Polymorphism

Where: `LibraryService` stores all members in a `List<LibraryUser>` and calls abstract methods on each element at runtime. The JVM dynamically dispatches to the correct subclass implementation.

Code Snippet (`LibraryService.java` — `returnBook`):

```
// member is a LibraryUser reference - actual type resolved at runtime
int daysLate = (int) daysBorrowed - member.getBorrowDurationDays();
double fineAmount = member.calculateFine(daysLate);

// In issueBook:
if (member.getBorrowedBooks().size() >= member.getMaxBooks()) {
    throw new BorrowLimitExceededException(...);
}
```

Explanation: The variable `member` is typed as `LibraryUser`. When the actual object is a `StudentMember`, `member.calculateFine()` invokes `StudentMember.calculateFine()` (fine rate \$1/day, 14-day window). When the object is a `FacultyMember`, the same call invokes `FacultyMember.calculateFine()` (\$0.50/day, 30-day window). `LibraryService` never checks the runtime type — this is runtime polymorphism in action.

3.4 Encapsulation

Where: All fields in `Book`, `LibraryUser`, and `Transaction` are declared private, with public getters and, where modification is intentional, public setters.

Code Snippet (`Book.java`):

```
public class Book implements Serializable {
    private String bookId; // cannot be accessed directly
    private boolean isAvailable;

    public boolean isAvailable() { return isAvailable; }
    // Only one path to mutate availability:
    public void setAvailable(boolean available) {
        this.isAvailable = available;
    }
}
```

Explanation: External code cannot set `isAvailable = true` arbitrarily — it must call `setAvailable()`, which is called exclusively by `LibraryService` inside the controlled `issueBook()` and `returnBook()` operations. This guarantees that availability changes only as part of a complete, validated transaction.

3.5 Exception Hierarchy

Where: The exception package defines a custom hierarchy rooted at `LibraryException` extends `Exception`.

Code Snippet (exception package):

```
// Base checked exception - all library errors are catchable via LibraryException
public class LibraryException extends Exception {
    public LibraryException(String message) { super(message); }
}
// Specific subtypes for precise error handling:
public class BookNotAvailableException extends LibraryException { ... }
```

```
public class BorrowLimitExceededException extends LibraryException { ... }
public class InvalidReturnException extends LibraryException { ... }
```

Explanation: Callers may catch `LibraryException` to handle all library errors uniformly, or catch individual subtypes to apply type-specific recovery logic. This mirrors standard Java exception design and makes `Main.java`'s try-catch block clean and extensible.

4. Implementation Highlights

4.1 Fine Calculation Logic

The `returnBook` method in `LibraryService` computes overdue fines using polymorphism and `ChronoUnit`:

```
long daysBorrowed = ChronoUnit.DAYS.between(openTx.getIssueDate(),
LocalDate.now());
int daysLate = (int) daysBorrowed - member.getBorrowDurationDays();
double fineAmount = member.calculateFine(daysLate);
```

Justification: The actual member subtype determines the allowed borrow duration and fine rate. Using `member.getBorrowDurationDays()` rather than a hardcoded constant means the same logic works for students (14-day window, \$1/day) and faculty (30-day window, \$0.50/day) without branching. `calculateFine()` guards against negative `daysLate` values, so on-time returns always produce a fine of 0.

4.2 Data Persistence with Java Serialization

`StorageUtil` wraps `ObjectOutputStream` and `ObjectInputStream` to persist the entire object graph:

```
public static void saveObject(Object obj, String filepath) {
    try (ObjectOutputStream oos =
        new ObjectOutputStream(new FileOutputStream(filepath))) {
        oos.writeObject(obj);
    } catch (IOException e) {
        System.err.println("Error saving: " + e.getMessage());
    }
}
```

Justification: By implementing `Serializable` in `Book`, `LibraryUser`, and `Transaction`, the entire state can be saved and restored in two method calls. The try-with-resources idiom ensures streams are always closed, even on exception, preventing resource leaks.

4.3 Transaction Tracking

Each issue operation creates a `Transaction` keyed by a UUID:

```
Transaction tx = new Transaction(
    UUID.randomUUID().toString(), bookId, memberId, LocalDate.now()
);
transactions.add(tx);
```

Justification: Transactions form an audit trail. When a book is returned, the service locates the open transaction (`returnDate == null`) to calculate the elapsed days. UUID ensures no collision between transaction identifiers across sessions.

4.4 Reservation Queue

```
private Queue<String> reservations = new LinkedList<>();

public void reserveBook(String memberId) {
    LibraryUser member = getMember(memberId);
    if (member != null) reservations.offer(memberId);
}
```

Justification: A Queue (FIFO) is the semantically correct data structure for reservations — members are served in the order they joined the wait list. LinkedList was chosen as the Queue implementation because the reservation list is short and requires no random access.

5. Testing & Error Handling

5.1 Test Cases

Test Case	Expected Outcome	Handled By
Issue book to valid member	Book marked unavailable; Transaction created	LibraryService.issueBook()
Issue already-issued book	BookNotAvailableException thrown	book.isAvailable() guard
Exceed student borrow limit (>3)	BorrowLimitExceededException thrown	member.getMaxBooks() check
Return book on time	Fine = \$0.00	calculateFine(daysLate ≤ 0) → 0
Return book 5 days late (Student)	Fine = \$5.00	StudentMember.calculateFine(5)
Return book 5 days late (Faculty)	Fine = \$2.50	FacultyMember.calculateFine(5)
Return book not borrowed by member	InvalidReturnException thrown	borrowedBooks.contains() guard
Issue book to non-existent member	IllegalArgumentException thrown	member == null check
Load state from corrupted .dat file	Graceful null fallback, fresh state	StorageUtil null check + re-init
Register same member ID twice	Both inserted; looked up by first match	List iteration; no ID uniqueness enforced

5.2 Edge Cases Handled

- daysLate ≤ 0 in calculateFine(): Both subclasses explicitly return 0.0 for non-positive values, preventing negative fines.
- Null book or member in issueBook(): A single combined guard (book == null || !book.isAvailable()) throws BookNotAvailableException before any state is mutated.
- No open transaction at return time: If openTx is null (data corruption scenario), the fine defaults to 0.0 and the return still succeeds.
- Empty data files on first run: StorageUtil.loadObject() returns null; LibraryService replaces null with a fresh collection, so the system initialises cleanly.

5.3 Failure Scenarios

- Disk write failure in `saveData()`: `IOException` is caught and logged to `stderr`; the in-memory state remains consistent for the current session.
- `ClassNotFoundException` on load: Indicates a serialised class was renamed or removed. `StorageUtil` catches it and returns null, triggering a fresh state.
- Concurrent modification: The current single-threaded CLI design does not face thread-safety issues, but the mutable `List<LibraryUser>` and `HashMap<String, Book>` would require synchronisation in a multi-threaded extension.

6. Git Workflow

The project was developed incrementally, with each commit representing a logical unit of work. The Git history reflects the natural progression from skeleton setup through model creation, service logic, exception handling, persistence, and final CLI integration.

```
PS D:\Java\term-ii-project-submission-adrijadutta-tech> git log --oneline
6c366a1 (HEAD -> main, origin/main, origin/HEAD) entry point for Library Management System
8bf2471 file-based data persistence utility
9e95818 core library operations and business logic
07b74ff base custom exception class
fe9db59 thrown on invalid return attempt
6f23fcf thrown when book is unavailable
d0edfab tracks borrow and return transactions
ae5eda8 Add student member model with borrow limits
7158fc7 Library user infos
e6cf017 faculty member infos
3f0ff4b Book infos are provided
09595b4 add deadline
59939a0 Initial commit
```

Name	Last commit message	Last commit date
..		
exception	base custom exception class	4 days ago
model	tracks borrow and return transactions	last week
service	core library operations and business logic	3 days ago
util	file-based data persistence utility	yesterday
.keep	Initial commit	27 days ago
Main.java	entry point for Library Management System	yesterday

7. Conclusion & Future Scope

7.1 Conclusion

The Modular Library Management System successfully demonstrates the application of core OOP principles in a non-trivial, real-world scenario. The abstract `LibraryUser` hierarchy, runtime polymorphism in fine calculation, custom checked exception hierarchy, and Java serialisation-based persistence together form a cohesive, extensible design. The system correctly handles all major use cases — issuing, returning, fine calculation — and gracefully reports errors via a clean exception model.

7.2 Future Scope

- **GUI Front-End:** Replace the Scanner-based CLI with a JavaFX or web-based interface for improved usability.
- **Database Persistence:** Replace Java serialisation with a relational database (e.g., SQLite via JDBC) to support concurrent access and richer queries.
- **Reservation Completion:** Extend reservations so that when a book is returned, the first member in the queue is notified automatically.
- **Admin vs. Member Roles:** Add role-based access control so that only administrators can add books or register members.
- **Search & Filter:** Allow searching books by title, author, or availability status.
- **Email Notifications:** Send overdue reminders to members approaching or past their return date.
- **Thread Safety:** Protect shared mutable state with `ReentrantReadWriteLock` to support concurrent clients in a server deployment.

8. Appendix

8.1. Initial Project Proposal

Modular Library Management System with Fine Calculation and Reservation Engine

Problem Statement

Small academic departments and private libraries often rely on manual or semi-digital systems to manage book circulation. This leads to inconsistent record keeping, incorrect fine calculations, difficulty tracking issued books, and inefficient reservation handling. Enforcing borrowing rules for different user categories (students, faculty, etc.) also becomes difficult without structured logic. This project proposes a well-designed object-oriented Java-based Library Management System that automates book issuing, returning, fine calculation, and reservation management. The system enforces borrowing policies programmatically, ensures data integrity through encapsulation, and handles exceptional cases such as issuing unavailable books or returning books that were never issued. The goal is a clean, modular, and defensible implementation demonstrating strong OOP principles and robust system design.

Target Users

- College department libraries
- Small institutional libraries
- Academic labs maintaining internal book collections

Core Features

- Add, update, and search books
- Register and manage members
- Book issue and return functionality
- Automated fine calculation based on due dates
- Reservation system using queue mechanism
- Persistent storage using file handling
- Custom exception handling for rule violations
- Transaction history tracking

OOP Concepts Used

Abstraction: Abstract class `LibraryUser` defining common attributes and borrowing rules. **Inheritance:** `StudentMember` and `FacultyMember` extend `LibraryUser` with different borrowing limits and fine policies. **Polymorphism:** Overridden `calculateFine()` method implementing different logic for each member type. **Exception Handling:** `BookNotAvailableException`, `InvalidReturnException`, `BorrowLimitExceededException`. **Collections:** `HashMap` for books, `ArrayList` for members, `Queue` for reservation management.

Proposed Architecture

The system follows a layered modular architecture consisting of a Model Layer (`Book`, `LibraryUser`, `Transaction` classes), Service Layer (business logic for issuing books, returning books, fine calculation, and reservation processing), Exception Layer (custom exception classes), and Utility Layer (file handling and data persistence). Clear package separation ensures maintainability, scalability, clean code organization, strong encapsulation, and low coupling within the system.

8.2. Review Mail (APPROVED_MINOR)



PCCCS495 Proposal Review – APPROVED_MINOR

From soumadip.biswas@ieminternational.ai <soumadip.biswas@ieminternational.ai>

Date Wed 3/11/2026 9:34 AM

To Adrija Dutta <Adrija.Dutta2024@iem.edu.in>

Dear Adrija Dutta,

Project: Modular Library Management System with Fine Calculation and Reservation Engine

Decision: APPROVED_MINOR

Score: 90

Strengths:

The proposal presents a clear, relevant, and well-defined problem statement for a Library Management System, explicitly identifying its target audience. The feature set is comprehensive and appropriate for a university-level Java OOP project, covering core functionalities like issue/return, fines, reservations, and persistence. A significant strength is the explicit commitment to strong OOP principles (Abstraction, Inheritance, Polymorphism, Encapsulation, Exception Handling, Collections) which are clearly aligned with the project's goals. The proposed layered architecture (Model, Service, Exception, Utility layers) demonstrates a sound understanding of modular design, separation of concerns, and maintainability, further bolstered by the mention of clear package separation, low coupling, and scalability. The inclusion of custom exception handling and file-based persistence adds practical depth.

Improvements:

1. While the OOP concepts are listed, providing a brief example of *how* inheritance/polymorphism will be applied (e.g., different types of `LibraryUser` with distinct borrowing policies, or various `FineCalculationStrategy` implementations) would further strengthen the design clarity.
2. Although file handling is acceptable for persistence, exploring a simple embedded database (like H2 or SQLite) could be considered as an alternative for a slightly more robust and scalable data management approach, enhancing the learning experience regarding database interaction.
3. The proposal does not explicitly mention a user interface. While not strictly necessary for demonstrating OOP principles, clarifying whether it will be a console-based application, a basic GUI (e.g., Swing/JavaFX), or purely a backend system would complete the user experience aspect.
4. Incorporating a plan for unit testing into the development process would further demonstrate a commitment to robust system design and good software engineering practices.

Regards,
Course Instructor